Waswa AI — Product Documentation

Audience: Product Manager, 3D Services Ltd
Prepared: 20 October 2026
Based on: NAVAS backend `navas-core-apis` (branch `waswa-ai`, commit `4a76477`) and OLIWA mobile app `oliwa-mobile` (branch `new-ui`, commit `e82ac21`)

This folder describes Waswa AI as it is **actually built** today. Every statement was checked against the source code, tests, configuration or existing project documents. Anything that could not be confirmed is labelled as such.

Documents

#	Document	What it answers
1	Product Overview	What Waswa is, who it is for, why it matters
2	Features and Capabilities	What it can and cannot do, with the status of each feature
3	User Guide	How customers use Waswa in OLIWA, and how staff use the review console
4	AI and Knowledge Base	How a question becomes an answer; where knowledge comes from; how documents are synced
5	APIs and Integrations	Endpoints, authentication, and connections to NAVAS, OLIWA and third parties
6	Security and Privacy	Permissions, account isolation, data handling, safeguards and gaps
7	Testing and Known Issues	Real test results, evaluation history, bugs and unverified areas
8	Recommendations and Roadmap	Prioritised improvements with acceptance criteria
9	Product Manager Handover	Key findings, release readiness, risks, and decisions needed

Short on time? Read the Product Manager Handover first.

Status labels used throughout

Label	Meaning
✅ **Verified**	Built, and confirmed by an automated test or evaluation run
🟢 **Built**	Present in the code and wired in, but not independently verified for this report
🟡 **Partial**	Built but incomplete, inconsistent, or not reachable by users
⛔ **Not built**	Mentioned in plans, prompts or the database, but no working code

Label	Meaning
📋 **Plan only**	Described only in strategy documents

How this was produced

- Source code, database migrations, prompts, tests and scripts in both repositories were read in full for Waswa-related areas.
- Backend unit tests that do not need a database were **run on 20 Oct 2026** (results in Testing).
- Tests that connect to the shared database, and the mobile (Flutter) tests, were **not run**. The database tests use a remote server, and Flutter is not installed on the documenting machine. Their last recorded results are reported instead.
- No application code was changed. No credentials, customer data or server addresses appear in these documents.

Source map

Area	Main files
Chat engine	`navas-core-apis/endpoints/assistant.py`, `endpoints/waswa_v2.py`
Account context	`endpoints/waswa_context.py`
Data access layer	`endpoints/waswa_data_api.py`, `waswa_data_auth.py`, `waswa_data_client.py`
Tools	`endpoints/waswa_fleet.py`, `waswa_products.py`, `waswa_knowledge.py`, `waswa_app_guide.py`
Corrections & review	`endpoints/waswa_answers.py`, `waswa_console.py`, `waswa_documents.py`
Database	`database/migrations/030–046_waswa_*.sql`, `database/manual/waswa_readonly_role.sql`
Prompts	`database/prompts/waswa_runtime_v2.md` … `v7.md`
Evaluation	`tests/waswa_eval/`, `scripts/waswa_eval_*.py`
Mobile UI	`oliwa-mobile/lib/screens/waswa_assistant_screen.dart`, `lib/widgets/waswa_chat.dart`, `waswa_copilot.dart`, `ask_waswa_report_sheet.dart`, `lib/providers/waswa_provider.dart`, `lib/services/assistant_service.dart`
App guide (Waswa's knowledge of the app)	`oliwa-mobile/docs/user-guide/`, `.claude/skills/kb-sync/`, `.github/workflows/kb-sync.yml`

1. Product Overview

← Index

What Waswa AI is

Waswa AI is the conversational assistant built into the NAVAS platform. A signed-in user types a question in plain language, such as "How many tokens do I have?", "Why is my truck offline?" or "How do I create a geofence?". Waswa answers using:

- **the user's own account data**: tokens, vehicles, trips, alerts, geofences and payments;
- **3D Services' approved documents**: manuals, procedures and the OLIWA app guide;
- **the product catalogue**: product capabilities and compatibility, but never prices;
- **corrections approved by 3D Services staff**, which take priority over everything else.

Technically, Waswa is a large language model (by default OpenAI GPT-4o, reached through the OpenRouter service). It is allowed to look things up only through a fixed set of read-only "tools" that the NAVAS backend controls. It cannot change anything in a customer's account.

Who it is for

User group	How they reach Waswa	What they get
OLIWA mobile app customers (fleet owners, managers)	Floating AI button on every screen, the side menu, Settings, Help & FAQ, Reports, and Token Billing	Answers about their own fleet and tokens, how to use the app, and report requests
3D Services staff (support, sales, operations)	The same chat, through the CMS and other surfaces (`surface` = `cms` , `oliwa_console`)	Answers from internal staff-only documents as well, source references, and the ability to look up any client's fleet
Waswa reviewers and approvers (internal)	Review console endpoints (`/assistant/console/*`), used by the CMS	Review flagged answers, write and approve corrections, manage the document library

The backend supports a CMS chat surface and the console. The CMS front-end itself is in a separate repository that was **not inspected** for this report.

Business value (as designed)

These are the intended benefits implied by the implementation. **None has been measured.** No usage, satisfaction, deflection or revenue metrics exist in the code or the repositories.

- **Self-service support.** Customers can answer "where is my vehicle / why is it offline / how many tokens are left" questions without calling support.
- **App adoption.** Step-by-step answers come from the official OLIWA app guide.

- **Consistent answers.** Staff corrections are reviewed by a second person before Waswa uses them, so one fix improves every future answer to that question.
- **Pricing protection.** Waswa never quotes prices or discounts; those questions are handed to the sales team.
- **Product discovery.** Waswa can recommend at most two suitable products, without prices. The Handover covers a conflict in this behaviour.

Strategy vs. reality

The document `database/knowledge/navas_waswa_ai_strategy_feb2026.md` (a February 2026 strategy note) describes a three-tier "AI cascade":

1. free rule-based handling for about 90% of work;
2. a self-hosted model;
3. a paid external model for under 1% of traffic.

It also describes caching, batching and token-billing for "external AI", with a projected cost reduction.

What is built is different:

Strategy item	Status
Rule-based tier	🟡 Only the pricing filter and some automatic data look-ups run without the AI model
Self-hosted model	⛔ Not built. A `router_tier` value of 2 is reserved in the database and never used
External model for most work	🟢 Every non-pricing question goes to one external model (GPT-4o by default), not a cheaper "mini" model
Caching, batching, fine-tuning	⛔ Not built
Charging customers tokens for AI use	⛔ Not built. There is no per-user AI cost accounting
Cost reduction of the stated size	📋 A projection in the strategy note; not measurable from the current system

Current maturity in one sentence

Waswa is a working, read-only question-answering assistant with real account isolation, a careful human review process for corrections, and an evaluation harness. It has **not yet been formally released** (the backend branch is unmerged), it has **no usage limits**, and several planned features (actions, smart routing, cost control) exist **only on paper**.

2. Features and Capabilities

← Index · Status labels: ✅ Verified · 🟢 Built · 🟡 Partial · ⛔ Not built · 📋 Plan only

2.1 What customers can ask about

Waswa answers by calling read-only "tools" (`navas-core-apis/endpoints/assistant.py:110-111`). Each tool is listed below with what it gives the customer.

Capability	What the user gets	Data used	Status
Token balance and tracking time left	Wallet tokens, packs awaiting use, time left (1 token = 1 hour), worded the same as the dashboard	Token accounts (account context)	✅ `tests/test_tracking_time_left.py` , `test_wallet_owner.py`
Find a vehicle	Match by plate, name, VIN, make or IMEI; asks "which one?" if ambiguous	Device registry (`unit_find`)	✅ `tests/test_waswa_fleet.py`
Vehicle status / "why is it offline?"	Last heartbeat, last position, billing status, pause rules, trip in progress	`unit_status`	✅ `test_waswa_fleet.py`
Fleet-wide liveness	Which vehicles are reporting or silent, and which could not be read (a failed read is never counted as "offline")	`fleet_liveness` , up to 120 units	✅ `tests/test_fleet_liveness.py`
Trips	Trips for a vehicle over up to 31 days; the same source as the Trips report	`unit_trips`	✅ `test_waswa_fleet.py`
Fleet activity	Trips and kilometres per vehicle over recent days	`fleet_activity`	🟢 (the model is told it covers 7 days by default; it actually covers 2, see Known Issues)
Places along a route	Places passed, optionally only where speeding	`unit_route_probe` + Google Places	🟢 Needs `GOOGLE_MAPS_API_KEY` ; capped at 25 look-ups

Capability	What the user gets	Data used	Status
Alerts	Recent alerts with severity and read state	`alerts_recent`	✅ `tests/test_alerts_tool.py`
Geofences	Zones, their shape, and which vehicles are inside now	`geofences_list`	✅ `tests/test_waswa_geofences.py` (wording only)
Payments standing	Counts and totals of recorded payments by status; explicitly "not a credit assessment"	Account context	✅ `tests/test_payment_standing.py`
Empty-account guidance	When there are no tokens or products, points to recorded payments without guessing what "pending" means	Account context	✅ `tests/test_zero_state.py`
How the OLIWA app works	Step-by-step guidance from 31 app-guide articles	`app_guide_search`	✅ eval questions; 🟡 guide is partly out of date (see Known Issues)
How things work / policy / billing	Answers from approved documents	`knowledge_search`	✅ eval questions; customers only see documents marked for "everyone"
Products and compatibility	Find, compare and check products against existing hardware. **No prices.**	Product catalogue tables	✅ `tests/test_product_search_words.py`, `test_waswa_ivms_lookup.py`
Product recommendations	Up to two suggested products when a customer describes a need; cheaper options (pause, auto-renew off, VEBA) when cost is raised	Prompt instruction	🟡 Conflicts with the system prompt, which says not to raise sales (see Handover)

Capability	What the user gets	Data used	Status
Pricing and discounts	Always a polite hand-off to sales; never reaches the AI model. Questions about the customer's *own* charges are still answered	Rule-based filter (also catches Swahili "bei gani", "gharama")	✅ `tests/test_pricing_gate.py`
Report requests (mobile)	The user describes a report in words; Waswa fills in the Reports form (type, dates, vehicles) and generates it	Chat endpoint + app-side parsing	🟡 Labelled **BETA** in the app; no automated test of the parsing
Token-buying help (mobile)	A separate "Ask AI" chat on Token Billing that turns package names in a reply into **Buy** cards	Chat endpoint	🟡 Not tested; ignores the user's Waswa ON/OFF switch

2.2 Conversation features

Feature	Status	Notes
Follow-up questions in the same thread	✅	The server keeps history and replays the last 10 messages; mobile tests cover the threading
Conversation resumes within 60 minutes	🟢	Same surface, last activity within 60 minutes (`waswa_data_api.py:209-242`)
"New conversation"	✅	Header ⋮ menu in the app
Conversation history list in the app	⛔	History lives in memory only; it is lost on sign-out or app restart
Archive / restore conversation	🟡	Backend console only (staff). No mobile UI. Conversations are never deleted
Screen awareness ("Context: Live Tracking")	🟢	23 app screens tell Waswa which screen the user is on
Suggested questions	🟢	4 quick chips + 6 suggestions sheet
Answer feedback (Helpful / Not helpful)	✅	"Not helpful" creates an item in the staff review queue
"Verified answer" badge	✅	Shown when a staff-approved correction was used

Feature	Status	Notes
ON/OFF switch	✅	The user can switch Waswa off; it is remembered on the device
Rich formatting (tables, links, lists)	🟡	Only **bold** and *italic* are styled; links are not tappable
Streaming (words appear as typed)	⛔	The whole answer arrives at once; up to 90 s wait
Voice, images, attachments	⛔	—
Languages other than English	⛔	No language handling. The pricing filter recognises two Swahili phrases only

2.3 Staff and back-office features

Feature	Status	Notes
Staff-only knowledge and source references	✅	Staff see internal documents and a "Source:" line
Look up any client's fleet (staff)	🟢	Staff can name a client in chat; customers are always pinned to their own account
Review queue (flags, pending corrections, re-checks, new documents)	🟢	`GET /assistant/console/queue`
Write a correction ("verified answer")	🟢	Draft → submit → approve. Currency amounts are refused
Two-person approval	✅	Nobody can approve their own correction (enforced in code **and** database). "Policy" topics need two approvers
Corrections expire for re-check	🟢	After 180 days, or when their source document changes
Document library: upload PDF, Word, Excel, Markdown, text	🟢	Max 25 MB; prices are automatically blanked out; no OCR for scanned PDFs
Document versioning and approval	🟢	A new version goes live only after approval; the old version keeps answering until then
Remove / restore a document	🟡	Works, but there is no way to roll back to a previous version (see Known Issues)
OLIWA app guide sync	🟢	Manual sync works; the automatic CI sync is **not active**
Read all customer conversations	🟢	Reviewers can see every tenant's conversations (see Security)
Preview what a question would match	🟢	`GET /assistant/console/match`
Audit trail	🟢	Every console change goes to a transparency log and a tamper-evident audit log

2.4 What Waswa cannot do

These limits are deliberate, by current design:

- **No actions.** Waswa cannot pause tracking, buy tokens, change settings, contact anyone, or immobilise a vehicle. All 15 tools are read-only.
- **No prices or discounts.** There is no price book connection (Odoo is not wired in).
- **No live push.** Waswa answers only when asked; it does not monitor events or send alerts.

2.5 Designed in the database but not built

Item	Where it exists	Status
Gated actions: about 30 sales/ops actions with approval levels (e.g. schedule demo, send reminder, approve discount)	`database/migrations/033_waswa_gated_actions.sql`, `endpoints/waswa_gating.py`	⛔ No code calls it. Waswa cannot perform any of these, even "automatic" level-0 ones. 11 sensitive actions have no approver assigned
Offers / opportunity tracking	`dll_waswa_offers` table (migration 030)	⛔ Never written
Customer class (Diamond…Bronze)	Migration 031	🟡 Column exists; never filled automatically
Local / cheaper model tier	`router_tier = 2`	⛔
Semantic (meaning-based) search	`embedding` columns (migration 035)	⛔ Reserved; search is keyword-based
Token burn rate	Account context	⛔ Always "not measured yet"
RBAC permissions "Can chat with Waswa", "Can access Waswa AI", "Can view AI cost", etc.	Migrations 013 and 018	⛔ Appear in the permissions catalogue but are **not enforced**: switching them off has no effect

3. User Guide

← Index

This guide describes the OLIWA mobile app as it is in the code on branch `new-ui` (commit `e82ac21`). The official customer-facing articles are in `oliwa-mobile/docs/user-guide/articles/ask-waswa.md` and `ask-waswa-for-a-report.md`. Where those articles differ from the app, it is noted in ⚠.

3.1 Who can use Waswa

- **Any signed-in OLIWA user.** The app has no role, plan or permission check for Waswa (`oliwa-mobile/lib/widgets/waswa_copilot.dart:131-148`).
- The backend decides what data the user may see, always based on their own account.

3.2 Ways to open Waswa

Where	What to tap
Any screen (except Reports)	The round floating **AI** button. It can be dragged out of the way
Dashboard	☰ menu → **Waswa AI** (sparkle icon)
Settings	The dark card **Help (Chat with Waswa)**
Settings → Help & FAQ	Banner "Still need help? Ask Waswa AI." → **Chat**
Reports → Generate	**Ask Waswa AI** card (opens the report assistant; see 3.5)
Token Billing → Buy	**Ask AI** chip (a separate token-buying assistant; see 3.6)

The first four all open the **same conversation**.

⚠ The guide says the AI button "floats over every screen". It is hidden on Reports.

3.3 Asking a question

1. Open Waswa. A greeting appears with four quick questions:
   - "How many tokens do I have?"
   - "Why is a vehicle offline?"
   - "How do tracking tokens work?"
   - "Explain my charges"
2. Tap a quick question, or type in the box **"Ask Waswa about..."** and tap the green send button.
3. For more ideas, tap the **sparkle** button (tooltip "Suggested questions"). It offers six topics: GPS tokens, balance, charges, buying tokens, auto-renew, pausing tracking. ⚠ The guide calls this the **+** button.
4. While Waswa works, "Waswa is typing..." is shown. Answers can take up to 90 seconds; in the last recorded evaluation answers averaged about 18 seconds.

5. Ask follow-ups normally. Waswa remembers the conversation.

Screen context. The header shows **Context: *screen name***, meaning Waswa knows which screen you came from (for example Live Tracking). This helps it give relevant answers.

3.4 After an answer

- **Helpful** (thumbs up) or **Not helpful** (flag) appear under each answer.
  - "Not helpful" shows *"Sent for correction — our team will fix this"* and puts the answer in the staff review queue.
- **Verified answer** chip: the answer used a correction approved by 3D Services staff. Tap it to see which approved question was matched.
- **Errors.** If the network fails, a red message such as *"Cannot reach the server. Check your internet connection."* appears with **Try again**. ⚠ The guide quotes a different error text that users will rarely see.

3.5 Asking Waswa for a report (BETA)

1. Go to **Reports** → **Generate** and tap **Ask Waswa AI**.
2. Describe the report, for example "Overspeeding for all trucks last week". You can also tap one of the **TRY** examples.
3. Tap **Fetch Report**. Waswa works out the report type, dates and vehicles.
   - Supported report types: trips, overspeeding, fuel, geozone, night driving, idling, parking.
   - If you don't say otherwise, it uses the last 7 days and all vehicles.
4. The Reports form is filled in and the report is generated. A message "Waswa: …" explains what was chosen.

Notes:

- This does not appear in the main chat history.
- It works even if Waswa is switched off.

3.6 Ask AI on Token Billing

On **Token Billing** → **Buy**, the **Ask AI** chip opens a separate assistant for choosing a token package. If its reply names a package, a **Buy** card appears that leads into the normal purchase flow.

Notes:

- This assistant has no feedback buttons.
- It also works when Waswa is switched off.
- ⚠ Waswa never quotes prices (the `buy-tokens` guide says so too), yet the starter suggestions are phrased around a budget. Users may expect a price.

3.7 Switching Waswa off and on

Tap the **ON/OFF** pill in the chat header.

- When off, an amber banner "*Waswa is switched off.*" appears with **Switch on**.
- Your choice is remembered on the device.

3.8 Starting over and privacy

- **New conversation.** ⋮ menu → **New conversation**.
- **Signing out** clears the conversation from the phone.
    - There is no history list in the app.
    - Conversations remain stored on the server (see Security).

3.9 What to tell customers Waswa cannot do

- It cannot quote prices or discounts. It will direct them to sales.
- It cannot make changes, such as pausing tracking, buying tokens or editing geofences. It explains how to do these in the app instead.
- It can only see the customer's own account.

3.10 Staff: reviewing and correcting Waswa (back office)

The console is available through the API (`/assistant/console/*`). The screens are provided by the CMS, which was not inspected for this report.

Who.

- Users holding `waswa.review` (reviewers) or `waswa.approve` (approvers).
- Super admins, system accounts and internal admins get these automatically.
- Customer accounts can never hold them.

Typical workflow:

1. **Queue.** Open the review queue. It shows flagged answers, corrections awaiting approval, corrections due for re-check, and new documents.
2. **Inspect.** Open the conversation to see the full messages, the data Waswa used ("evidence"), and the customer's feedback.
3. **Correct.** Draft a *verified answer*.
    - Currency amounts are not allowed.
    - Text touching money, approvals or account actions is classed as "policy" and needs **two** approvers. Routine fixes need one.
4. **Submit** the draft. Drafts that were never submitted can be deleted.
5. **Approve.** A *different* person approves it. Waswa starts using it at once for matching questions.
6. **Resolve** the original flag.

7. **Upkeep.** Approved answers come back for re-check after 180 days, or when the document they rely on changes.

Documents:

1. **Upload** (PDF/Word/Excel/Markdown/text, ≤25 MB). Choose an authority level (2–6), a document type, and an audience: "staff" (default) or "everyone", meaning customers too.
2. A **different person** approves it. Admins may self-approve, and this is logged.
3. To update a document, upload a **new version**. The old version keeps answering until the new one is approved.
4. **Remove** hides a document (reversible). See Known Issues for the rollback limitation.

OLIWA app guide sync. Done by a developer using the `/kb-sync` skill in the `oliwa-mobile` repo; see AI and Knowledge Base.

4. AI and Knowledge Base

← Index

4.1 How a question becomes an answer

All chat goes through **one endpoint**, `POST /assistant/chat` (`endpoints/assistant.py:775`). For each question:

```
User question (OLIWA app / CMS)
   │
   ▼
1. Check sign-in (JWT) ──────────────── no token → 401
   │
   ▼
2. Load the user's account context ──── tokens, products, units, alerts, payments
   Open or resume the conversation      (built on the server; anything the app sends is ignored)
   Decide audience: customer or staff
   │
   ▼
3. Pricing filter (no AI involved) ──── price/discount question → fixed hand-off reply, logged
   │
   ▼
4. Build instructions for the AI:
   active system prompt + account context + approved "verified answers"
   + rules (e.g. use the app guide for mobile how-to; never turn "unavailable" into zero;
   always state the date range; never mention price)
   │
   ▼
5. Pre-fetch obvious fleet data (vehicle named → look it up first)
   │
   ▼
6. AI model (OpenRouter, default GPT-4o) chooses tools ◄──┐
   Tools run on the NAVAS server, scoped to the account ──┘ up to 6 rounds
   │
   ▼
7. Clean-up: remove "Source:" lines for customers, strip anything secret-looking
   (optional v2 mode: check the answer and repair once)
   │
   ▼
8. Save the answer, the evidence used, model, prompt version and timing
   Return reply + message ID (for feedback) + verified-answer references
```

Key settings (`endpoints/assistant.py`):

Setting	Value
Answer length limit	1,200 tokens
Temperature (randomness)	0.3
Tool rounds per question	up to 6
History replayed	last 10 messages
Timeout per AI call	45 s (the app waits up to 90 s)
Network retries	2, with short back-off
Fallback model	**none**

If the AI provider key is missing, chat returns "not configured" (HTTP 503). This happened in production once, as recorded in `oliwa-mobile/docs/WORK_REPORT_2026-09-10.md`.

Forcing look-ups. For any question that is not small talk, the AI is *required* to call at least one tool before answering. This stops it answering account questions from memory (`assistant.py:304-321`; ✅ `tests/test_tool_forcing.py`).

v1 and v2. These are two modes of the same endpoint, not two products.

- v1 is the default.
- v2 adds a "wrong-corpus" filter (it drops non-mobile steps for mobile how-to questions) and an answer check with one repair attempt.
- The mode is chosen by the `WASWA_AGENT_VERSION` server setting, **or by any caller per request**. See Security.

4.2 The system prompt

- The instructions given to the AI are versioned in the database (`dll_waswa_prompts`). Only one version is active at a time.
- Versions are managed with `scripts/waswa_prompt.py` (load, activate, roll back) and take effect without a restart.
- The newest file in the repo is `database/prompts/waswa_runtime_v7.md`.
- **Which version is live in production cannot be confirmed from the repository.** Evaluation records refer to prompt version 6 and later 9. Database version numbers do not necessarily match file names.

4.3 Where Waswa's knowledge comes from

Waswa draws on five sources, in this order of trust:

Source	Trust	Who sees it
Staff-approved corrections ("verified answers")	Highest. Injected before the AI starts	Filtered by audience

Source	Trust	Who sees it
Live account and fleet data (tools)	High: system of record	Own account only (staff: any)
Product catalogue	High (authority level 1)	Everyone; no prices
Approved documents	Weighted by authority level 2–4	Customers: "everyone" documents only
OLIWA app guide (31 articles)	Level 2, audience "everyone"	Everyone

Document search (retrieval)

- **Keyword search, not meaning-based.** Search uses PostgreSQL full-text search. Paraphrased questions can miss relevant text. Semantic (vector) search is planned but not available, because pgvector is not installed (`endpoints/waswa_knowledge.py:10-28`).
- **Chunking.** Documents are split by heading into pieces of up to about 1,800 characters. Each piece keeps its heading path.
- **Search order.** Exact phrase first, then all words, then at least half of the distinctive words.
- **Ranking.** Relevance is multiplied by document authority, so an official manual outranks a user story on an equal match.
- **Filters.** Only approved, active, current-version documents are searched, and only those allowed for the caller's audience. Authority levels 5–6 are never quoted.
- **Volume.** Up to 6 pieces (maximum 7,000 characters) per search. App-guide search returns up to 3 whole articles.
- **Caveats to the AI.** The search adds warnings when, for example, it found only loose matches or only specification-level documents.

Current document library

Twelve documents are listed in `database/knowledge/manifest.json` :

Authority	Documents
2 (official)	Sales Procedures (v26), NAVAS CMS User Manual
3	Customer Journey Maps (2024), Token Billing Strategy, Ecosystem Vision & Scope, Waswa AI Strategy
4 (specifications)	OLIWA, CMS and Marketplace user stories; UKO Dashboard KPIs
5 (never quoted)	Two internal strategy documents

Points the PM should know:

- The 12 documents were loaded by script **without an audience setting**, so they default to **staff only**. Unless someone changed them in the console, customers cannot get answers from them; only the app guide is customer-visible. The live database state was not checked.
- The authority levels 2–6 are marked "proposed, unconfirmed" in migration 035.
- The file labelled "UKO" user stories is a mislabelled copy of the OLIWA stories.

- Some documents are old (journey maps from 2024). The Sales Procedures lack the discount threshold figure.
- Prices are deliberately removed from every document at upload.

4.4 Keeping the app guide in sync

The OLIWA app guide lives in the mobile repository (`oliwa-mobile/docs/user-guide/articles/`, 31 articles) so that it is updated alongside the app.

How it reaches Waswa:

1. A developer runs the `/kb-sync` skill (`oliwa-mobile/.claude/skills/kb-sync/`). It finds articles affected by code changes, updates them, and validates them: no prices, no internal names, all referenced screens exist.
2. The script `kb_sync.py publish` sends the articles to `POST /assistant/console/app-guide/oliwa-mobile/sync`, signed in as a staff user with `waswa.review`.
3. The server compares each article with what it holds and creates, updates, leaves alone or (optionally) removes it. All of this happens in one database transaction.
4. New versions wait for approval, unless an admin publishes directly.

Automatic sync (CI). A GitHub workflow (`oliwa-mobile/.github/workflows/kb-sync.yml`) is designed to do this on every push to `main`: Claude updates the articles, then they are published. **It is not active today**:

- The workflow file exists only on the `new-ui` branch. GitHub only runs push workflows from the default branch, so it has never run.
- The secrets it needs (an AI key and a NAVAS staff login) could not be verified.
- It compares only the last commit, so changes spread across several commits can be missed.

Current state. Publishing has been done manually. The last publish (14 Oct 2026, commit `4b7e9b1`) was self-approved by an admin. Since then, nine app screens have changed and four new screens have no article (alert analytics, fleet status, geofence presence, token analytics). All 31 articles are marked "verified from code", **not checked on a device**, although the guide's README says otherwise.

4.5 Learning from mistakes ("training")

Waswa is **not retrained**. "Training" means curating what it reads:

1. A customer taps **Not helpful**.
2. Staff review the answer and write a correction.
3. A second person approves it.
4. Waswa finds the approved correction by keyword match (up to 3 per question) and places it at the top of its instructions.

Corrections are re-checked after 180 days, or when their source document changes.

4.6 Evaluation harness

`tests/waswa_eval/` holds 21 test questions:

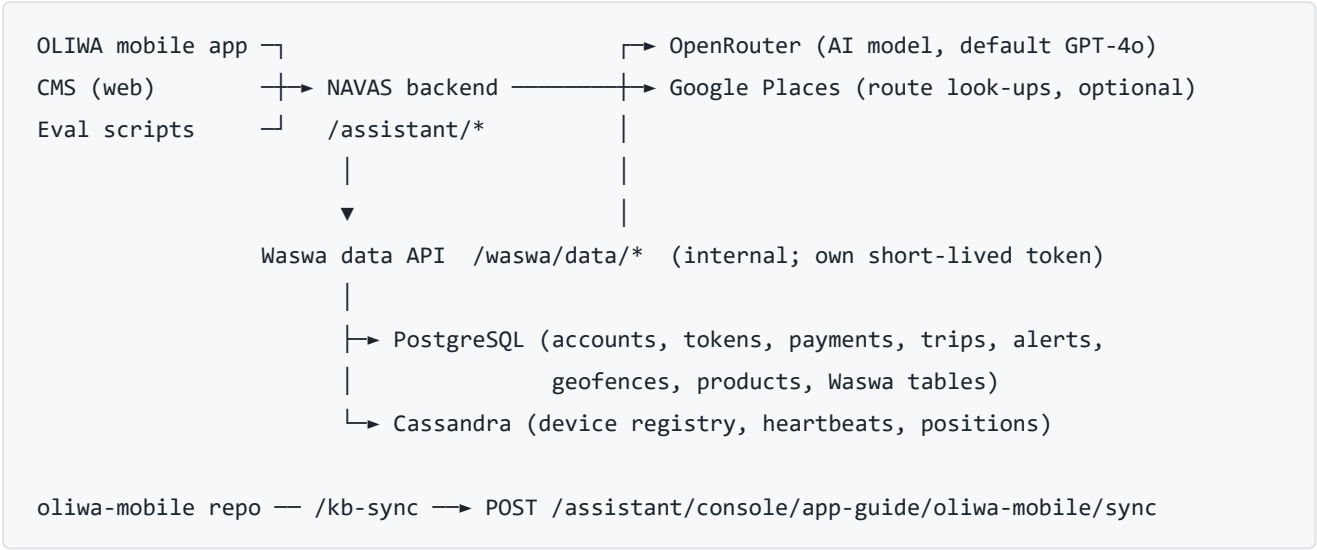
- 12 taken from real user complaints
- 3 pricing rules
- 6 document facts

`scripts/waswa_eval_run.py` asks each question and scores the answer with fixed rules, for example "must call the right tool", "must not contain a price" and "every number must come from the data". Results are recorded in `tests/waswa_eval/runs/` and a local MLflow tracker. See Testing for the results.

5. APIs and Integrations

← Index

5.1 System picture



How the routes are wired:

- The routes are registered in `endpoints/__init__.py`.
- Chat and console routes have no URL prefix. Several console routes are added with `add_url_rule` in `endpoints/waswa_console.py` and `waswa_app_guide.py`, rather than with decorators.
- The data API uses the prefix `/waswa/data`.

5.2 Authentication

Who	Method	Details
App and CMS users	`Authorization: Bearer <JWT>`	From `/users/auth`; access token valid 30 min by default, refresh token 7 days (`config.py:53-54`). The mobile app refreshes once on a 401, then signs out
Waswa → data API	Separate **data token**	Created on the server for each chat turn and never sent to the app. Valid 10 min. Signed with its own key and bound to issuer, audience and scope, so it cannot be swapped with a user token (`endpoints/waswa_data_auth.py`; ✅ `tests/test_waswa_data_api.py`)
App-guide sync script	Staff JWT	Uses either a token or a staff username/password from environment variables (names: `NAVAS_KB_TOKEN`, `NAVAS_KB_USERNAME`, `NAVAS_KB_PASSWORD`)
NAVAS → OpenRouter	API key	`OPENROUTER_API_KEY` (server environment)

5.3 Endpoints

Chat and customer endpoints

Any signed-in user can call these. No extra permission is needed.

Method	Path	Purpose	Source
POST	`/assistant/chat`	Ask a question	`assistant.py:775`
POST	`/assistant/feedback`	Rate an answer: `good`, `unhelpful` or `wrong` (only the conversation owner or a reviewer)	`waswa_console.py:168`
GET, POST	`/assistant/context`	Show what Waswa knows about the caller's account (diagnostic)	`assistant.py:691`
GET, POST	`/assistant/knowledge`	Show what a document search would return (diagnostic)	`assistant.py:723`

Chat request (as sent by the mobile app, `oliwa-mobile/lib/services/assistant_service.dart:85-97`):

```
{ "data": { "message": "How many tokens do I have?",
            "surface": "mobile",
            "conversation_uid": "<optional, to continue>",
            "module": "<optional screen name>" } }
```

Chat response (main fields):

Field	Meaning
`reply`	The answer text
`message_uid`	The answer's ID, used for feedback
`conversation_uid`	The conversation's ID, sent back to continue it
`verified_answers[]`	The staff-approved corrections that were used
`evidence`	The data Waswa looked at
`tool_rounds`	How many tool rounds the turn took
`truncated`	Set when the answer hit the length limit
`model`	The AI model's name

The server also accepts `agent_version` (`v1` or `v2`). The app ignores and does not send `context` and `history`, which the server ignores too.

Review console: `waswa.review`

Staff only.

Method	Path	Purpose
GET	`/assistant/console/summary`	Badge counts and the caller's permissions
GET	`/assistant/console/queue`	Review queue
GET	`/assistant/console/conversations`, `/conversations/<uid>`	All conversations; full transcript with evidence
GET, POST	`/assistant/console/answers`, `/answers/<uid>`	List, create or edit corrections
POST	`/assistant/console/answers/<uid>/submit`, `/retire`, `/delete`	Correction lifecycle (delete: unsubmitted drafts only)
POST	`/assistant/console/feedback/<uid>/resolve`	Close a flag
GET	`/assistant/console/sources`, `/sources/<uid>`	Document library
POST	`/assistant/console/sources/upload`	Upload a document (multipart, ≤25 MB)
GET	`/assistant/console/authority-levels`	Trust levels
GET, POST	`/assistant/console/match`	Preview which corrections a question matches
GET	`/assistant/console/app-guide/<app>`	App-guide articles and status
POST	`/assistant/console/app-guide/<app>/sync`	Sync the app guide (`dry_run`, `prune`, `publish` options)

Approval console: `waswa.approve`

Method	Path	Purpose
POST	`/assistant/console/answers/<uid>/decide`, `/confirm`	Approve or reject a correction; confirm at re-check
POST	`/assistant/console/sources/<uid>/review`, `/audience`, `/remove`, `/restore`, `/details`	Document approval and management
POST	`/assistant/console/conversations/<uid>/archive`, `/restore`	Archive or restore a conversation

Internal data API: data token only

Not for apps.

Path	Purpose
`GET /waswa/data/caller`, `/context`, `/prompt`	Audience, account context, active prompt

Path	Purpose
`POST /waswa/data/conversations` ; `GET/POST /conversations/<uid>/messages` ; `POST /conversations/<uid>/transparency`	Conversation storage (owner-checked)
`POST /waswa/data/messages/<uid>/evidence`	Evidence trail
`POST /waswa/data/answers/match`	Verified-answer matching
`GET /waswa/data/knowledge/sources` ; `POST /knowledge/search`	Documents
`POST /waswa/data/tools/<name>`	Runs one of the 15 tools

By default the chat engine calls these **inside the same server process**. Setting `WASWA_DATA_API_URL` makes it call them over HTTP instead (`endpoints/waswa_data_client.py`).

API specification status

`docs/waswa_api_openapi.json` is **out of date**:

- It is dated to migration 041 (22 Sep 2026).
- It documents 21 of the 35 `/assistant` method/path pairs.
- It covers none of the 12 data-API routes.
- It omits the `module` and `agent_version` chat fields.

This folder's table above is the current reference.

5.4 Integration with OLIWA (mobile)

Item	Detail
Endpoints used	`/assistant/chat` , `/assistant/feedback` (`oliwa-mobile/lib/core/constants/api_constants.dart:115-116`)
Timeout	90 s for chat; 20 s for feedback
Streaming	No; one complete JSON response
State	`WaswaProvider` (app-wide) keeps the current conversation in memory
Screen context	23 screens send their name as `module`
Not used by the app	Conversation archive and history endpoints; `agent_version`

5.5 Integration with NAVAS data

Waswa reads only. The tables it reads are listed in `database/manual/waswa_readonly_role.sql` and in AI and Knowledge Base:

- **PostgreSQL:** account relay, token accounts and registry, device subscriptions, event notifications (alerts), payment logs, pause rules, trips, geofences, location registry, product catalogue.

- **Cassandra:** device basic data, heartbeats ("pulse"), positions.

Waswa's own tables (migrations 030–046) hold:

Area	Tables
Conversations and messages	`dll_waswa_conversations`, `dll_waswa_messages`
Evidence	`dll_waswa_evidence`
Transparency log	`dll_waswa_transparency_log`
Prompts	`dll_waswa_prompts`
Documents and chunks	`dll_waswa_sources`, `dll_waswa_chunks`
App guide	`dll_waswa_app_guide`
Feedback	`dll_waswa_feedback`
Corrections and approvals	`dll_waswa_answers`, `dll_waswa_answer_approvals`
Gated actions (unused)	`dll_waswa_gated_actions`

5.6 Third-party services

Service	What is sent	Configuration
OpenRouter (to OpenAI GPT-4o by default)	System prompt, the user's account context (tenant name, token balance, payment summary, products, vehicles), up to 10 previous messages, tool results (plates, IMEIs, trips, locations, alerts)	`OPENROUTER_API_KEY`, `OPENROUTER_MODEL`, `OPENROUTER_SITE_URL`, `OPENROUTER_SITE_NAME`
Google Places	Route coordinates (route look-ups only)	`GOOGLE_MAPS_API_KEY`
GitHub Actions + Anthropic (planned CI for the guide)	App source code and guide articles	Not active

5.7 Configuration reference

Variable **names** only, as listed in `.env.example`.

Variable	Purpose	Default
`OPENROUTER_API_KEY`	AI provider key. If empty, chat is off	—
`OPENROUTER_MODEL`	Model	`openai/gpt-4o`
`WASWA_AGENT_VERSION`	`v1` or `v2`	`v1`
`WASWA_DATABASE_URL`	Optional restricted database login	falls back to the main DB login

Variable	Purpose	Default
`WASWA_DATA_TOKEN_SECRET`	Data-token signing key	derived from `JWT_SECRET`
`WASWA_DATA_TOKEN_TTL_SECONDS`	Data-token lifetime	600
`WASWA_DATA_API_URL`	Run the data API over HTTP	in-process
`GOOGLE_MAPS_API_KEY`	Route place look-ups	off

6. Security and Privacy

← Index

This section describes protections and gaps factually. It contains no credentials and no exploit details.

6.1 Permissions

Who can use chat. Every signed-in user. No permission is checked (`endpoints/assistant.py:775+`).

Waswa permissions that are enforced (`endpoints/waswa_answers.py:42-116`):

Permission	Unlocks
`waswa.staff_knowledge`	"Staff" audience: internal documents, source references, **and the ability to read any client's fleet through chat**
`waswa.review`	The review console; also staff audience
`waswa.approve`	Approving corrections and documents, managing the library, archiving conversations; also staff audience

Who gets them:

- **No role receives them by explicit grant.** The seed scripts create the permissions but assign them to nobody.
- They are given **automatically** to super admins, system accounts, and admin/sysadmin roles on internal (non-customer) accounts.
- **Customer accounts** (`client` , `customer` , `customer_tracker`) are always stripped of every `waswa.*` permission, even if a role ticks it.

Permissions that look active but are not. The RBAC catalogue contains "Can chat with Waswa", "Can access Waswa AI", "Can approve AI action", "Can view AI cost" and others (migrations 013, 018). **No code checks them.** An administrator who un-ticks "Can chat with Waswa" will see no effect.

6.2 Account data isolation (customers)

This is the strongest part of the design.

Safeguard	How	Evidence
The account is always taken from the sign-in token	Nothing in the request can change whose data is read	`endpoints/waswa_context.py` , `waswa_data_api.py`
The app cannot inject context or history	`context` and `history` sent by a client are ignored	`assistant.py:803-806`

Safeguard	How	Evidence
The AI cannot widen its own access	The server strips `scope`, `audience`, `client_uid` and similar arguments from tool calls before they run	`waswa_data_api.py:66, 389-426`; ✅ tested
Another customer's vehicle is invisible	Returns "not found", never "belongs to someone else"	`waswa_fleet.py`; ✅ `tests/test_waswa_fleet.py`
Another person's conversation is invisible	Ownership checked on every read/write; returns 404	✅ `tests/test_waswa_data_api.py`
The AI never touches the database directly	All data goes through the data API with a separate 10-minute token	✅ `tests/test_waswa_data_api.py`
Customers see only "everyone" documents and corrections	Audience filter on every search	`waswa_knowledge.py:130-141`
Prices never reach customers	Pricing questions are stopped before the AI; currency amounts are blocked in corrections and redacted from documents	✅ `tests/test_pricing_gate.py`
Secrets never appear in answers	Replies are scrubbed of credential-like text	✅ `tests/test_reply_scrub.py` (65 checks)
SQL injection	All database queries use parameters; Cassandra uses prepared statements	Code review

6.3 Staff access (by design, but worth a decision)

1. **Staff can read any client's fleet through chat.** This includes vehicles, positions, trips and alerts. Anyone with *any* `waswa.*` permission counts as staff (`waswa_data_api.py:154-162`). The permission's description mentions only documents.
2. **Reviewers can read every customer's conversations**, across all tenants, with no tenant filter (`waswa_console.py:273-366`).
3. **Admins can bypass the second-person check for documents.** They can self-approve an upload and publish the app guide in one step. This is logged as `self_approved`, but there is no second person. Corrections, by contrast, can never be self-approved.

6.4 Privacy and data handling

- **Storage.** Every question and answer is stored word-for-word in `dll_waswa_messages`.
- **Retention.** There is **no retention or deletion policy**. Archiving hides a conversation but keeps it.
- **Data sent to a third party.** Each chat turn sends the user's account context and fleet data to **OpenRouter**, which forwards it to the model provider (OpenAI by default). This includes tenant name, token balance, payment summary, vehicle plates, IMEIs, locations and trips.
  - The request sets **no "do not retain / do not train" options**.
  - No data-processing agreement was found in the repository.
- **Route look-ups.** These send coordinates to Google Places (when enabled).

- **Server logs.** Logs record errors and tool failures, not message text.
- **Signing out** on mobile clears the conversation from the phone.

6.5 Abuse and cost protection

Protection	Status
Rate limit on chat	⛔ **None**. Only login, refresh and forgot-password are rate-limited (`app.py:29-39`)
Message length limit	⛔ None
Per-user or per-account AI spend cap	⛔ None
Per-answer limits	🟢 1,200-token answers, 6 tool rounds, 10 history messages, 45 s timeout
Upload limit	🟢 25 MB, fixed file types
Prompt-injection defence	🟡 Structural only: the AI cannot widen scope, pricing bypasses the AI, and documents must be approved. There is no filter on user text or document content
`agent_version` switch	🟡 Any caller can choose v1 or v2 per request (meant for testing)

6.6 Other technical findings

#	Finding	Location
1	The restricted "read-only" database role for Waswa exists as a script but is **not active** until `WASWA_DATABASE_URL` is set. Until then Waswa uses the main database login. The script has also **drifted from the code**: applying it as written would break draft deletion, app-guide sync and geofence look-ups	`database/manual/waswa_readonly_role.sql`
2	Credential rotation is recorded as **not started**. Five of six secrets can be recovered from git history. The AI provider key is not among them	`database/manual/a2_credential_rotation.md`
3	Console permission errors return internal detail (the caller's role and permissions, raw error text). Chat responses include the model name and internal "unknown context" reasons	`waswa_console.py:82-97` , `assistant.py:1231-1261`
4	The logout blacklist check lets tokens through if the database is unavailable ("fails open")	`endpoints/jwt_utils.py:115-120`
5	The default allowed web origins include localhost addresses, unless production overrides `CORS_ORIGINS`	`config.py:92-98`
6	Data tokens can be reused within their 10-minute life. The data API is reachable from outside even though it is normally called in-process	`waswa_data_auth.py` , `endpoints/__init__.py`

#	Finding	Location
7	The customer-account check used by Waswa looks at account type only; the platform-wide check also looks at role	`waswa_answers.py:84-88` vs `globals.py:346-369`

6.7 What is tested and what is not

Area	Status
Data-token separation; conversation ownership; fleet scoping; secret scrubbing; pricing gate; signed-in gate	✅ Tested
Console permission matrix (review vs approve; customer admin refused)	⛔ No tests
Staff cross-client fleet access	⛔ No tests
Stripping of `waswa.*` from customer accounts	⛔ No tests
Tenant scoping in console views	⛔ No tests

7. Testing and Known Issues

← Index

7.1 Automated tests run for this report (20 Oct 2026)

Backend tests that do **not** need a database or the AI model were run on the current code (`waswa-ai` @ `4a76477`). Some files are written as standalone scripts, so they were run with `python tests/<file>.py` ; the others were run with pytest.

Test file	What it checks	Result
`test_waswa_v2.py`	v2 agent end-to-end with a fake model: wrong-corpus guard, retrieval recovery, "live" claims	✅ 69 checks passed
`test_waswa_fleet.py`	Whose fleet a call may read, route-probe cost caps, trip building	✅ 45 passed
`test_reply_scrub.py`	Secrets removed from replies	✅ 65 passed
`test_payment_standing.py`	Payment totals; withholds totals when amounts are unreadable	✅ 37 passed
`test_zero_state.py`	Empty-wallet guidance never guesses or mentions price	✅ 26 passed
`test_fleet_liveness.py`	A failed read is never reported as "offline"	✅ 18 passed
`test_wallet_owner.py`	The right account's token balance is used	✅ 16 passed
`test_eval_runner.py`	Evaluation runner HTTP mode and crash recovery	✅ 16 passed
`test_tool_forcing.py`	Account questions force a data look-up	✅ 14 passed
`test_signed_in_gate.py`	Anonymous callers get 401	✅ 10 passed
`test_waswa_ivms_lookup.py`	"iVMS product" resolves correctly; empty product filter is widened	✅ 7 passed
`test_product_search_words.py`	Loose product search ("fuel")	✅ 5 passed
`test_pricing_gate.py`	Customers never get prices; staff may ask about the discount *process*	✅ 4 passed
`test_alerts_tool.py`	Alert summary shape and severity	✅ 4 passed
`test_tracking_time_left.py`	Time left matches the dashboard	✅ 3 passed
`test_waswa_geofences.py`	Geofence shape wording	✅ 3 passed

Total: 16 Waswa-related test files, 342 checks, 0 failures. Other non-Waswa backend tests run at the same time also passed.

`test_permission_loader.py` and `test_blacklist_cache.py` (platform auth, not Waswa-specific) could not be collected, because a library was missing in the local test environment. This is an environment issue, not

a test failure.

Not run for this report

Tests	Why not run	Last recorded result
`test_waswa_app_guide.py` (11 tests: guide sync, versioning, publish rules, permissions)	Connects to the shared remote database	Recorded as **failing** in the local pytest cache (16 Oct 2026), most likely because no database was reachable at that time. **Unverified**
`test_waswa_data_api.py` (12 tests: token separation, ownership, isolation)	Needs the database **and** a live AI call	Recorded as failing to load (16 Oct 2026). **Unverified**
OLIWA Flutter tests: `waswa_copilot_test.dart` (~20), `waswa_chat_format_test.dart` (2), Waswa parts of `services_test.dart` and `integration_test.dart`	Flutter SDK not installed on the documenting machine	No recorded result found. **Unverified**

> **Note:** five backend test files are standalone scripts. A plain `pytest` run does not pick them up, so they only run when someone runs them by hand. No CI configuration for the backend was found.

7.2 Answer-quality evaluation (recorded results)

Taken from `tests/waswa_eval/runs/*.json` and the local MLflow log. These runs were **not repeated** for this report.

Date	Pass rate (15 questions)	Notes
9 Oct	73% → 80%	Early runs; scoring rules changed several times
10 Oct	0% / 20%	Not quality failures: authentication error (401) against production, then an AI-provider outage (502)
10 Oct	67% → 93%	Steady improvement; v1 and v2 compared side by side
11 Oct (latest)	**100% (15/15)** for v1 and v2	v2 average answer time about 18.5 s, 95th percentile about 22 s

How to read this:

- Scoring is rule-based (right tool used, no price, figures traceable to data). It does **not** judge helpfulness or tone. The "no dead-end answers" check has never been scored.
- The latest result comes from **one run per version**. The evaluation tooling itself warns that one run is not a measurement, and answers vary between runs.
- **Six document-fact questions** added on 14 Oct have **never been run**. The latest result therefore covers 15 of 21 questions.

- **No evaluation since 11 Oct.** Changes from 12–17 Oct (dashboard, alerts, iVMS, deletion/archiving) have not been evaluated.
- The question set's expected answers are marked "draft — correct by hand".
- Baseline from real usage (`scripts/waswa_eval_build.py`): of 92 early answers, 14 received user verdicts (8 wrong, 3 unhelpful, 3 good), and 78 were never reviewed. This is the only real-usage quality figure in the repository.

7.3 Known issues and bugs

Severity is our assessment: **High** affects customers or data protection; **Medium** affects correctness or operations; **Low** is cosmetic or internal.

#	Issue	Severity	Evidence
K1	No rate limit, message-length limit or spend cap on chat, so cost exposure is unbounded	High	`app.py:29-39`, `assistant.py:789`
K2	Customer data sent to OpenRouter/OpenAI with no retention/no-training settings, and no retention policy for stored conversations	High	`assistant.py:554-594`
K3	The app guide is out of date: 9 screens changed since the last publish, 4 new screens have no article, and nothing has been checked on a device	Medium–High	`oliwa-mobile/docs/user-guide/`
K4	Prompt v7 says "do not raise sales opportunities"; the code tells the AI to upsell/downsell on every turn	Medium	`database/prompts/waswa_runtime_v7.md:140`, `assistant.py:936-951`
K5	RBAC switches "Can chat with Waswa" and similar do nothing	Medium	Migrations 013/018; no code checks them
K6	Removing a bad approved document version leaves **no** live version (the old one cannot be restored or re-uploaded)	Medium	`waswa_documents.py:447-453, 492-498`
K7	Automatic app-guide sync (CI) has never run	Medium	`oliwa-mobile/.github/workflows/kb-sync.yml` exists only on `new-ui`
K8	Restricted database role not applied, and out of step with the code	Medium	`database/manual/waswa_readonly_role.sql`
K9	API specification (OpenAPI) is out of date	Medium	`docs/waswa_api_openapi.json`

#	Issue	Severity	Evidence
K10	Report assistant and Token Billing "Ask AI" ignore the user's Waswa OFF switch	Medium	`oliwa-mobile/lib/widgets/ask_waswa_report_sheet.dart`, `token_billing_screen.dart`
K11	Likely UI bug: opening Waswa from the side menu, Settings or FAQ may leave the floating button visible on top of the chat, and tapping it opens a second copy. **Inferred from code; confirm on a device**	Low–Medium	`oliwa-mobile/lib/widgets/waswa_copilot.dart`
K12	Tool descriptions given to the AI are wrong: `fleet_activity` claims a 7-day default (actually 2); knowledge searches claim 4 results (actually 6 and 3)	Low	`waswa_fleet.py:785, 1094`; `waswa_knowledge.py:65, 492`
K13	In v2 mode, the "unsourced numbers" check never runs in production (needed data is not passed)	Low	`assistant.py:1198-1222`, `waswa_v2.py:385`
K14	Mobile chat shows tables, lists and links as raw text; links not tappable	Low	`oliwa-mobile/lib/widgets/waswa_chat.dart:16-38`
K15	User guide text differs from the app ("+" button, error message, "every screen")	Low	`oliwa-mobile/docs/user-guide/articles/ask-waswa.md`
K16	Orphan upload folders left on disk by rolled-back uploads and tests	Low	`waswa_documents.py:352-362`
K17	Many commit messages are vague ("updates", "local tests"), which makes changes hard to audit	Low	git history

7.4 Unverified functionality

These are present in the code, but nothing in the repository proves they work end-to-end:

- The CMS chat and review console screens (the CMS repository was not inspected).
- A real AI answer on a physical phone: mobile acceptance test #15 is marked 🟡 manual (`oliwa-mobile/docs/ACCEPTANCE_TESTS.md:40`).
- App-guide sync against the current database (its test needs the DB).
- Report-request parsing and Token Billing Ask AI (no tests).
- Which system prompt version is live in production.
- Whether any of the 12 core documents have been made visible to customers.
- Production configuration: AI key present, CORS origins, restricted DB role.

8. Recommendations and Roadmap

← Index

These are recommendations, not existing features. Priorities follow customer impact, risk and effort.

- **P0**: before wider release
- **P1**: next phase
- **P2**: later

No effort or business-impact figures are claimed; they need estimation by the team.

P0: before wider customer release

R1. Usage limits and cost visibility

Why: Chat has no rate limit, length limit or spend cap (K1). One user or script could run up AI costs.
Acceptance criteria:

- Chat is limited per user, for example N messages per minute and M per day (values to be set by management). Excess requests get a friendly message and HTTP 429.
- Messages over an agreed length are refused before reaching the AI.
- Daily AI usage (requests and tokens) per account is recorded and viewable by staff.
- An automated test proves the limit triggers.

R2. Privacy settings and a retention policy

Why: Customer fleet and payment data go to a third-party AI provider on default terms, and conversations are kept forever (K2). **Acceptance criteria:**

- Management has approved a policy that names the provider, the data categories sent, and the retention period.
- OpenRouter requests use the provider's "no data retention / no training" options, or an approved alternative.
- A scheduled job deletes or anonymises conversations older than the agreed period. A test proves it.
- The customer privacy notice mentions Waswa.

R3. Resolve the upsell contradiction

Why: The prompt forbids sales suggestions while the code demands them (K4). **Acceptance criteria:**

- A management decision is recorded.
- The code and the active prompt say the same thing.
- One evaluation question checks the chosen behaviour.

R4. Bring the app guide up to date, and verify it on a device

Why: Waswa reads the guide to customers. Stale steps produce wrong answers (K3, K15). **Acceptance criteria:**

- Every article matches the current `new-ui` build.
- The four new screens have articles, or are listed as undocumented on purpose.
- At least the top 10 articles by expected usage are walked on a device and marked `verified: device`.
- The guide is re-published and approved by a second person.

R5. Make the "Can chat with Waswa" switch real, or remove it

Why: Administrators believe they can turn Waswa off for a role; they cannot (K5). **Acceptance criteria:**

- Either `/assistant/chat` refuses users without the permission (HTTP 403, with a clear app message) and a test proves it,
- or the unused permissions are removed from the catalogue.

R6. Merge and release discipline

Why: 34 commits of Waswa work are not on `main`. **Acceptance criteria:**

- `waswa-ai` is reviewed and merged.
- Database migrations 030–051 are confirmed applied in production.
- The production configuration checklist is signed off: AI key, CORS origins, prompt version.

P1: next phase

R7. Apply the restricted database role

Acceptance criteria:

- The grant script is updated to match the code (K8).
- `scripts/waswa_grant_check.py` passes.
- `WASWA_DATABASE_URL` is set in production.
- Draft delete, app-guide sync and geofence look-up still work.

R8. Review staff access scope

Acceptance criteria:

- Management decides who may read any client's fleet through chat, and who may read all customers' conversations (see Handover).
- Permissions and their descriptions match that decision.
- Tests cover the permission matrix.

R9. Turn on automatic app-guide sync

Acceptance criteria:

- The workflow is on the default branch, with its secrets configured.
- A push that changes a screen opens a guide-update PR.
- Publishing requires a second-person approval (no auto-publish).
- The workflow compares the whole push, not only the last commit.

R10. Regular quality evaluation

Acceptance criteria:

- The evaluation runs on every merge, or weekly, against all 21+ questions, at least 3 times per run.
- Results are tracked over time.
- A release is blocked if the pass rate falls below an agreed threshold.
- The draft expectations are confirmed by a domain expert.
- A helpfulness / "no dead-end" judgement is added.

R11. Document version rollback

Acceptance criteria:

- An approver can restore the previous version of a document in one action (K6).
- A test covers remove-new-version → previous version answers again.

R12. Mobile chat polish

Acceptance criteria:

- Lists, tables and links render properly, and links are tappable (K14).
- All Waswa entry points respect the OFF switch (K10).
- The floating button is hidden while the chat is open (K11).
- Widget tests cover these.

R13. Up-to-date API specification

Acceptance criteria:

- The OpenAPI file lists every `/assistant` route and the chat fields `module` and `agent_version` (K9).
- It is generated or checked automatically.

P2: later

#	Recommendation	Acceptance criteria (summary)
R14	Meaning-based (semantic) search	Paraphrased versions of evaluation questions retrieve the same documents as the originals
R15	Conversation history in the app	The user can see and reopen past conversations; archive from the app

#	Recommendation	Acceptance criteria (summary)
R16	Streaming answers	First words appear within an agreed time
R17	Gated actions (e.g. schedule a demo, send a reminder)	Approval levels and approvers are confirmed by management for all ~30 actions; each action is logged; nothing runs without the defined approval
R18	Cost tiering from the strategy document	Measured share of questions answered without the paid model; cost per conversation tracked
R19	Language support (Swahili and others)	Agreed languages answered correctly in the evaluation set
R20	Fix tool descriptions and v2 number check	Descriptions match code defaults; the v2 unsourced-number check receives context data in production (K12, K13)

9. Product Manager Handover

← Index

9.1 Key findings

1. **Waswa works as a read-only assistant.** It answers questions about a customer's own tokens, vehicles, trips, alerts, geofences and payments, how the OLIWA app works, and which products fit a need. It does this with GPT-4o (through OpenRouter) and 15 controlled look-up tools.
2. **Customer data isolation is well designed and tested.**
    - The account always comes from the sign-in token.
    - The AI cannot widen its own access.
    - Other customers' data is invisible.
    - Prices are never given.
    - 342 automated checks passed on 20 Oct 2026.
3. **Quality is improving but thinly measured.**
    - The last recorded evaluation (11 Oct) passed 15 of 15 questions. That was one run, with rule-based scoring.
    - Six newer questions and a week of code changes have not been evaluated.
    - Real-usage data is limited: of 92 early answers, only 14 were rated, and 8 of those as wrong.
4. **There is a solid human-review loop.** Customer "Not helpful" flags feed a staff queue. Corrections need a second approver (two for policy topics) and are re-checked every 180 days.
5. **Much of the strategy is not built.**
    - No actions: the ~30 "gated actions" exist only as a database table.
    - No cheaper model tier, no caching, no AI cost billing, no semantic search.
6. **Waswa's knowledge of the app is out of date.** The app guide has not kept pace with the app (9 changed screens, 4 new ones). The automatic sync has never run.

9.2 Release readiness

Area	Readiness	Notes
Core chat (mobile)	🟢 Functional	Working in code; real-phone AI answer still a manual acceptance item
Data isolation	🟢 Good	Tested
Pricing protection	🟢 Good	Tested
Answer quality	🟡 Promising, not proven	One 100% run; no recent evaluation; no helpfulness measure
App-guide accuracy	🟡 Needs update	See R4
Cost control	🔴 Missing	No rate limit or spend cap
Privacy and retention	🔴 Undecided	Data goes to a third-party AI on default terms; conversations kept indefinitely

Area	Readiness	Notes
Permissions / admin controls	🟡 Misleading	"Can chat with Waswa" switch does nothing
Code merged to main	🔴 No	`waswa-ai` is 34 commits ahead of `origin/main`
Documentation (API)	🟡 Out of date	OpenAPI file stale; this folder is current

Overall assessment:

- **Suitable for a controlled pilot** with selected customers, provided the AI key and production configuration are confirmed.
- **Not yet ready for general release.** That requires at minimum R1 (usage limits), R2 (privacy decision), R3 (upsell decision), R4 (guide refresh) and R6 (merge and configuration sign-off) from the Roadmap.

9.3 Outstanding risks

Risk	Likelihood / impact	Mitigation
Unbounded AI costs from heavy or abusive use	Possible / High	R1
Customer data handled by a third party without an agreed policy	Already happening / High (legal and trust)	R2
Wrong how-to answers from a stale app guide	Likely / Medium	R4, R9
Inconsistent sales behaviour (upsell vs no-sales rule)	Already happening / Medium	R3
Admins believe they can disable Waswa per role	Likely / Medium	R5
Staff with any Waswa permission can read any client's fleet and all conversations	By design / Medium (insider risk)	R8
AI provider outage = Waswa down (no fallback model)	Has happened (502s on 10 Oct; missing key in production in Sep) / Medium	Add a fallback model and monitoring
Credentials still in git history (rotation "not started")	Present / High, platform-wide	Complete `database/manual/a2_credential_rotation.md`
A bad document version cannot be rolled back	Possible / Medium	R11

9.4 Decisions needed from management

1. **Sales behaviour.** Should Waswa recommend products (upsell and cheaper options) to customers now, or not until a later phase? Today the prompt and the code disagree.
2. **AI data policy.** Which customer data may be sent to an external AI provider, under which terms (no retention, no training, region), and how long should conversations be kept?
3. **Usage limits.** What message limits per user, per day, and what monthly AI budget? Should AI usage eventually consume customer tokens, as the strategy suggests?
4. **Staff access scope.** Who may:
    - look up any client's fleet through Waswa?
    - read all customers' conversations?

    Should admins be allowed to self-approve documents and app-guide publishes?
5. **Document visibility.** Which of the 12 internal documents (e.g. the CMS manual, token billing strategy) should customers be able to get answers from? Today they are staff-only by default. Confirm the proposed authority levels 2–6.
6. **Gated actions roadmap.** Should Waswa ever take actions? If so, who approves the 11 sensitive actions that currently have no approver (immobilise, suspend account, billing adjustment, etc.), and what are the discount thresholds?
7. **Release scope.** Pilot first (which customers, success measures) or general release? Which success metrics should be tracked (none exist today)? Suggested candidates:
    - share of answers rated Helpful
    - share of conversations with no support follow-up
    - average answer time
    - cost per conversation
8. **App-guide publishing.** Allow automatic publishing from CI, or always require a human approver?

9.5 Suggested next steps (first two weeks)

1. Decide items 1–3 above.
2. Merge `waswa-ai`, then confirm migrations and production configuration (R6).
3. Implement usage limits (R1) and the provider privacy settings (R2).
4. Refresh and device-check the app guide (R4), then re-run the full evaluation 3× and record results (R10).
5. Run a pilot with a small group of customers, reviewing the "Not helpful" queue weekly.